

GIT

The Visual Student Handbook

Understand the flow. Practice the commands.
Answer confidently in interviews.



REMEMBER

Git is not an acronym for 'Global Information Tracker.' It is simply the name Linus Torvalds chose for the tool.

Built for DevOps learners moving from fundamentals toward 3-5 years of interview-level understanding.

Why do teams need Git?

Begin with the real problem: many developers changing the same project over time.

THE PROBLEM

Without version control

Files become final.zip, final2.zip and nobody knows which version is correct. Changes are overwritten and rollback is manual.

THE SOLUTION

With Git

Every meaningful change becomes a traceable version. Teams can branch, review, merge, compare and restore work.

CORE DEFINITION

What is Git?

Git is a free, open-source, distributed version control system created by Linus Torvalds in 2005. It tracks file changes, stores versions as commits, and helps teams collaborate safely.

TERM 1

VCS

A Version Control System records file revisions so we can view history, compare versions and restore earlier work.

TERM 2

SCM

Source Code Management is the broader practice of managing source changes, branches, releases and team workflows. Git is the main VCS used within that process.

INTERVIEW ANSWER

Git is a distributed VCS. Each developer normally has the project and its complete history locally, which enables fast offline commits, branching, merging and reliable collaboration through remotes.

The Git journey of one change

If students understand these four locations, most Git commands become easy.



STEP 1

Working directory

The files currently checked out on your machine. You create, edit and delete here.

STEP 2

Staging area

A preparation area where you select the exact snapshot for the next commit.

STEP 3

Local repository

The .git database containing commits, branches, tags and history on your machine.

STEP 4

Remote repository

A shared Git repository hosted elsewhere, commonly on GitHub, GitLab or Bitbucket.

INTERVIEW ANSWER

The staging area is useful because it lets me create a precise and logically focused commit instead of committing every change present in my working directory.

Your first Git project

A small lab that shows the complete local lifecycle.

TRY IT IN YOUR LAB

\$ git --version

Confirm Git is installed.

\$ git config --global user.name 'Mustafa'

Set the author name.

\$ git config --global user.email 'mail@example.com'

Set the author email.

\$ mkdir demo && cd demo

Create and enter a practice folder.

\$ git init

Create the hidden .git repository.

\$ echo 'hello git' > app.txt

Create a working file.

\$ git status

Observe the untracked file.

\$ git add app.txt

Move its snapshot to staging.

\$ git commit -m 'Add app file'

Record the staged snapshot locally.

COMMON CONFUSION

git add does not save history

It updates the staging area. You can still change what goes into the commit.

COMMON CONFUSION

git commit is local

A commit does not automatically reach GitHub. Sharing happens with git push.

INTERVIEW ANSWER

git commit creates a commit object from the staged snapshot and advances the current branch reference. The commit contains metadata, a parent reference and a pointer to the project tree.

Inspect before you commit

Experienced engineers review changes before they save or push them.

git status

→

git diff

→

git add

→

git diff --staged

→

git commit

TRY IT IN YOUR LAB

\$ git status

Show branch, staged, unstaged and untracked files.

\$ git diff

Review unstaged line-by-line changes.

\$ git add app.txt

Stage only the intended file.

\$ git diff --staged

Review exactly what the next commit will contain.

\$ git commit -m 'Validate login input'

Create a focused commit with a meaningful message.

AVOID

Weak message

updated file

PREFER

Strong message

Validate login input before authentication

REMEMBER

Small commits are easier to review, test, revert and troubleshoot. One commit should represent one logical change.

Read history and repair the latest commit

Git history tells the story of how the project evolved.

TRY IT IN YOUR LAB

\$ git log

Full commit history.

\$ git log --oneline

Compact commit IDs and messages.

\$ git log --graph --oneline --decorate --all

Visual branch history.

\$ git log --follow -- app.txt

Follow one file's history across renames.

\$ git show

Inspect one commit and its patch.

EDIT THE LATEST COMMIT

Amend = replace the latest commit

Use `git commit --amend` to update the latest message, add forgotten staged changes, or correct the author. Git creates a new commit ID.

AMEND EXAMPLES

\$ git commit --amend -m 'Better message'

Replace the latest message.

\$ git add missed.txt && git commit --amend --no-edit

Add forgotten content and keep the old message.

WATCH OUT

Avoid amending a commit already shared with teammates. It rewrites history and changes the commit ID.

Undo without panic

The correct undo command depends on where the mistake currently exists.



REWRITE LOCAL HISTORY

git reset

Moves the branch pointer. --soft keeps changes staged, default mixed keeps them unstaged, and --hard discards tracked changes.

SAFE FOR SHARED HISTORY

git revert

Creates a new commit that reverses an older commit. Existing history stays visible and auditable.

TRY IT IN YOUR LAB

\$ git restore app.txt

Discard unstaged tracked changes.

\$ git restore --staged app.txt

Unstage but keep working changes.

\$ git reset --soft HEAD~1

Remove latest local commit; keep changes staged.

\$ git reset HEAD~1

Remove latest local commit; keep changes unstaged.

\$ git revert

Create a new reverse commit.

INTERVIEW ANSWER

I use reset mainly for unpublished local history. If a bad commit has already been pushed to a shared branch, I prefer revert because it preserves history and does not disrupt collaborators.

WATCH OUT

git reset --hard can permanently discard tracked working changes. Verify status and the target commit before using it.

Branches: parallel work without copying folders

A branch is a lightweight movable pointer to a commit.



TEAM STORY

Real-time scenario

The production-ready code remains on main. A developer creates feature/login, works independently, pushes it, opens a pull request and merges only after review and tests pass.

TRY IT IN YOUR LAB

\$ git branch

List local branches.

\$ git switch -c feature/login

Create and switch to a feature branch.

\$ git switch main

Return to main.

\$ git branch -m old new

Rename a branch.

\$ git branch -d feature/login

Delete a fully merged local branch.

\$ git branch -D feature/login

Force-delete an unmerged branch.

\$ git branch recovered

Recreate a deleted branch at a known commit.

REMEMBER

The default branch is configurable. Many repositories use main; older repositories often use master.

Merge the branch or pick one commit?

Choose based on whether you need the complete branch or one selected change.

BRING THE BRANCH

git merge

Combines another branch into the current branch. It may fast-forward or create a merge commit.

BRING ONE CHANGE

git cherry-pick

Applies one selected commit as a new commit on the current branch.

REAL-TIME SCENARIO

Production example

A critical payment fix exists on develop, but release must go out from main today. Cherry-pick only the tested fix commit instead of merging every unfinished develop change.

TRY IT IN YOUR LAB

\$ git switch main

Choose the receiving branch.

\$ git merge feature/login

Merge the complete feature branch.

\$ git cherry-pick

Apply one selected commit.

\$ git merge --abort

Cancel a conflicted merge.

\$ git cherry-pick --abort

Cancel a conflicted cherry-pick.

INTERVIEW ANSWER

Cherry-pick replays the selected patch on the current branch and creates a new commit, usually with a different ID because the parent and context are different.

Merge conflicts: Git needs a human decision

A conflict is not a failure. Git is asking which final content is correct.

CAUSE

Why it happens

Two branches change the same lines differently, or one side deletes a file that the other side modifies.

HUMAN INTENT

What Git cannot know

Git understands text changes, but it cannot decide the correct business requirement.

git status

→

open file

→

choose content

→

git add

→

test + commit

TRY IT IN YOUR LAB

\$ git status

Find unmerged paths.

\$ git diff

Inspect conflict sections.

\$ vim app.txt

Choose or combine the correct content and remove markers.

\$ git add app.txt

Mark the file resolved.

\$ run tests

Validate behavior before completing the merge.

\$ git commit

Complete the merge.

REMEMBER

Never delete conflict markers blindly. Understand both versions, confirm uncertain logic with the authors and run tests.

Merge vs rebase

Both integrate changes, but they tell the project story differently.

SAFE SHARED HISTORY

MERGE: preserve the roads

Keeps existing commit IDs and branch topology. It may create a merge commit.

CLEAN LOCAL HISTORY

REBASE: redraw one road

Replays commits on a new base to create a linear history. Replayed commits receive new IDs.

feature commits



git rebase main



resolve if needed



linear history

TRY IT IN YOUR LAB

\$ git switch feature/login

Move to the feature branch.

\$ git rebase main

Replay feature commits on top of main.

\$ git add resolved-file

Stage a conflict resolution.

\$ git rebase --continue

Continue replaying commits.

\$ git rebase --abort

Restore the original feature branch.

INTERVIEW ANSWER

Merge preserves branch history; rebase rewrites commits to create a linear history. I rebase unpublished feature work when appropriate, but I avoid rebasing commits teammates already use.

WATCH OUT

Golden rule: do not rebase shared public history unless the team explicitly coordinates the rewrite.

Stash, ignore and tag

Three small features that solve very different daily problems.

CONTEXT SWITCH

STASH: pause unfinished work

Temporarily stores tracked changes so you can switch tasks without creating a half-finished commit.

CLEAN REPOSITORY

.gitignore: keep noise out

Defines patterns for untracked logs, builds, local settings and secrets that should not enter version control.

TRY IT IN YOUR LAB

\$ git stash push -u -m 'WIP login'
Store tracked and untracked work.

\$ git stash list
List saved work.

\$ git stash apply
Apply latest and keep it stored.

\$ git stash pop
Apply latest and drop it on success.

\$ git stash drop stash@{2}
Delete one stash.

\$ git rm --cached .env
Stop tracking a file but keep the local copy.

RELEASE MARKER

Tag = name an important commit

Tags mark stable points such as v1.0.0. Annotated tags contain metadata and a message, so they are preferred for formal releases.

TAG A RELEASE

\$ git tag -a v1.0.0 -m 'Release 1.0.0'
Create an annotated release tag.

\$ git push origin v1.0.0
Share the tag with the remote.

Git is the engine; GitHub is the collaboration platform

Git works locally. GitHub hosts Git repositories and adds team workflows.

LOCAL TOOL

Git

Distributed version control: commits, branches, merges, tags and local history.

CLOUD PLATFORM

GitHub

Repository hosting plus pull requests, reviews, issues, permissions and CI/CD integrations.

INTERVIEW FAVORITE

What is origin?

origin is only a conventional remote name. It is normally created by git clone, but you can use another name and configure multiple remotes such as origin and upstream.

TRY IT IN YOUR LAB

\$ git remote add origin

Connect a local repository to a remote.

\$ git remote -v

Inspect remote names and URLs.

\$ git remote set-url origin

Change origin's URL.

\$ git remote rename origin company

Rename a remote.

\$ git remote remove origin

Remove only the local remote configuration.

Clone, fetch, pull and push

These four commands move repository data between local and remote locations.

FIRST COPY

CLONE

Create a new local repository from a remote, download history, check out a branch and configure origin.

SAFE INSPECTION

FETCH

Download new objects and update remote-tracking branches without changing your current branch.

DOWNLOAD + INTEGRATE

PULL

Fetch and then integrate the selected remote branch by merge or rebase, depending on configuration.

SHARE YOUR WORK

PUSH

Upload local objects and request an update to a remote branch or tag.

TRY IT IN YOUR LAB

\$ git clone

Create a local clone.

\$ git fetch origin

Refresh origin's remote-tracking data.

\$ git fetch origin main

Fetch a selected branch.

\$ git pull origin main

Fetch and integrate origin/main.

\$ git push -u origin feature/login

Push and configure upstream tracking.

\$ git push origin --delete old-branch

Delete a remote branch.

INTERVIEW ANSWER

Fetch downloads remote changes without modifying my current branch. Pull performs a fetch and then integrates the selected remote branch, so I often fetch first when I want to inspect changes safely.

A real DevOps team workflow

See how the commands connect from ticket assignment to production-ready merge.

1	Clone	<code>git clone</code>
2	Branch	<code>git switch -c feature/login</code>
3	Build	<code>edit + test locally</code>
4	Review	<code>git status && git diff</code>
5	Commit	<code>git add + git commit</code>
6	Sync	<code>git fetch origin</code>
7	Update	<code>git rebase origin/main</code>
8	Share	<code>git push -u origin feature/login</code>
9	Validate	<code>pull request + CI + review</code>
10	Merge	<code>merge approved code</code>

REMEMBER

Before pushing: verify the branch, inspect staged changes, run tests, confirm no secrets are committed and use a meaningful message.

Interview rapid revision

Strong answers explain behavior, safety and a real situation - not only a definition.

QUESTION

What is HEAD?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

HEAD identifies the currently checked-out branch or commit. It normally points indirectly through the active branch reference.

Add a real project example if the interviewer asks further.

QUESTION

Why use staging?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

To build a precise, reviewable snapshot instead of committing every working change.

Add a real project example if the interviewer asks further.

QUESTION

Reset vs revert?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

Reset rewrites a branch pointer and suits unpublished work; revert creates a new inverse commit and suits shared history.

Add a real project example if the interviewer asks further.

QUESTION

Merge vs rebase?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

Merge preserves topology; rebase rewrites commits onto a new base for linear history.

Add a real project example if the interviewer asks further.

Interview rapid revision - continued

Finish with four practical questions that commonly reveal real Git understanding.

QUESTION

Fetch vs pull?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

Fetch downloads without integrating; pull fetches and then merges or rebases.

Add a real project example if the interviewer asks further.

QUESTION

Git vs GitHub?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

Git is the VCS; GitHub hosts Git repositories and adds collaboration features.

Add a real project example if the interviewer asks further.

QUESTION

Why was push rejected?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

Common reasons include branch protection, missing permission or a non-fast-forward remote update.

Add a real project example if the interviewer asks further.

QUESTION

Why small commits?

Think for five seconds, then answer in one clear sentence.

STRONG ANSWER

They are easier to review, test, revert, cherry-pick and troubleshoot.

Add a real project example if the interviewer asks further.

REMEMBER

Best interview structure: define the concept, explain what Git changes internally, state the safety rule, and finish with one project example.